

Inserting Text

Description	Keystroke	Function	Note
Inserting Text - Insert quote char, Greek letter - File/Directory name - Software License, lice, spdx - To-do note - Date/Time - Automatic Time Stamping - Automatic Copyright note - Commented lines Smart Dash mode Smartparens mode - Text tempo skeletons Yasnippet See also: 🔗 Abbreviations	The commands described in this table insert specialized text at point (cursor) location. This includes: - Customization to control automatic insertion of time stamp, update of copyright notice. Simple command based text insertions: - PEL specialized commands to insert formatted text like time stamps, file path , file name, copyright notice, Greek letters, commented lines, etc...  The lice  activated by pel-use-lice, and spdx activated by  pel-use-spdx user options, can be used to insert open source licence text.  The smart-dash external package  activated by pel-use-smart-dash, used to automatically convert dash into underscore when typing.  The smartparens external package  activated by pel-use-smartparens to provide automatic insertion of balanced block pairs for code. Specialized template-based text insertion: - PEL tempo skeletons based templates for generic & specialized boilerplate file sections: file, class, function header, document section header.  The yasnippet external package  activated by pel-use-yasnippet to insert code from predefined snippets.  The yasnippet-snippets external package  activated by pel-use-yasnippet-snippets which provides a large amount of snippets. Hydra-based insertion of Greek letters:  The hydra external package.  activated by pel-use-hydra and pel-activate-hydra-for-greek.		
Last updated on:	2026-09-19	See also T Templates	
Open this PDF file. See also: 🔗 Help/Info	<code><f11> i <f1></code> <code><f11> y <f1></code> <code><f11> _ <f1></code>	(pel-help-pdf & optional OPEN-WEB-PAGE)	Open the 🔗 Inserting Text local PDF. With C-u , M-- , M- -1 or C- -1 prefix: open the remote GitHub hosted PDF instead. If the pel-flip-help-pdf-arg user-option is set it's the other way around.
🔗 Customize PEL Text Insertions control	<code><f11> i <f2></code>	(pel-customize-pel & optional OTHER-WINDOW)	Customize PEL text insertion support: lice, smart-dash, smartparens, tempo, time-stamp, yasnippet. Also pel-activate-f9-for-greek . If OTHER-WINDOW is non-nil (use C-u) , display in other window.
🔗 Customize Emacs Text Insertions control	<code><f11> i <f3></code>	(pel-customize-library & optional OTHER-WINDOW)	Customize Emacs text insertion support: lice, smart-dash, tempo, time-stamp, yasnippet
Insert <i>quoted</i> char	C-q	(quoted-insert ARG)	Read next input character and insert it. Useful for inserting control characters.
	- With argument, insert ARG copies of the character. - If the first character you type is an octal digit, the sequence of one or more octal digits you type is interpreted to specify a character code. Any character that is not an octal digit terminates the sequence. If the terminator is a RET, it is discarded; any other terminator is used itself as input and is inserted.		
Insert Greek Letter See also: 🔗 Input Method  In macOS you can also: add Greek as a Keyboard Input Sources and temporary select it to enter Greek characters.	<code><f9></code>  <code><f6></code> 	Insert a greek letter: type <code><f9></code> followed by a key in [a-zA-Z] range inserts the Unicode character for the equivalent Greek letter. Examples: <code><f9></code> a inserts α <code><f9></code> b inserts β <code><f9></code> A inserts Α <code><f9></code> B inserts Β <code><f9></code> 1 inserts λ	
	The insertions work everywhere insert is allowed, including in minibuffer. Use <code><f9></code> C-h or which-key mode and type <code><f9></code> to see all keys.  The <code><f9></code> key binding is only available when the pel-activate-f9-for-greek user-option is turned on. The <code><f6></code> g binding is always available.  It's not a command bound to a key: it's an additional set of bindings added to Emacs key-translation-map .		
	<code><f7></code> G	Start the Greek letter Hydra . - After typing <code><f7></code> G type one of the Meta letter keys in the hydra to insert a Greek character. - Type any other character to insert them, latin letters, digits, punctuation characters, and Meta-char to inter the greek character, etc... - In terminal mode the cursor keys may not work because they are often encoded using Esc keys with is mapped to Meta. Use C-b, C-f, C-n and C-p instead. - While this hydra is active, use <code><f11></code> u u for undo. - Exit the hydra by typing <code><f7></code> - The key bindings are shown by the hydra:	
 Requires the hydra external package.  activated by pel-use-hydra .  You must also set pel-activate-hydra-for-greek to t to activate this hydra.	[M-a]: α, [M-b]: β, [M-c]: χ, [M-d]: δ, [M-e]: ε, [M-f]: φ, [M-g]: γ, [M-h]: η, [M-i]: ι, [M-j]: ϕ, \[M-k]: κ, [M-l]: λ, [M-m]: μ, [M-n]: ν, [M-o]: ο, [M-p]: π, [M-q]: θ, [M-r]: ρ, [M-s]: σ, [M-t]: τ, \[M-u]: υ, [M-w]: ω, [M-x]: ξ, [M-y]: ψ, [M-z]: ζ, [M-A]: Α, [M-B]: Β, [M-C]: Χ, [M-D]: Δ, [M-E]: Ε, \[M-G]: Γ, [M-H]: Η, [M-I]: Ι, [M-J]: Φ, [M-K]: Κ, [M-L]: Λ, [M-M]: Μ, [M-N]: Ν, [M-O]: Ο, [M-P]: Π, \[M-Q]: Θ, [M-R]: Ρ, [M-S]: Σ, [M-T]: Τ, [M-U]: Υ, [M-W]: Ω, [M-X]: Ξ, [M-Y]: Ψ, [M-Z]: Ζ, [<f7>]: ca\ncel.		
Insert file/directory name	The following commands insert the name of the file or directory of the current file or file in specified window.		
Change filename root for the current buffer.  See also:  PEL Environment Variables	<code><f6></code> <code><f4></code> f	(pel-set-insert-filename-root)	Change the root directory that is stripped off the absolute path of the file name inserted by pel-insert-filename .
	This change is buffer specific and overrides the default value imposed by the pel-insert-filename-root user option or the value of the PEL_INSERT_FILENAME_ROOT environment variable. The value is taken in the following priority order (starting with the highest priority): value set by this command , PEL_INSERT_FILENAME_ROOT environment variable value, pel-insert-filename-root user option		
Customize to-do note format	<code><f11> i <f4></code> f	(pel-customize-insert-filename)	Open the customize buffer to change the 'pel-insert-filename-root' user-option.
Insert, at point, name of: - current filename by default - name of file in window identified by command numeric argument - Append line number of point in visited file - If file is in user home, use ~ at the beginning	<code><f11> i f</code> <code><f6> f</code>	(pel-insert-filename & optional N USE-TILDE DIR-ONLY)	Insert the file name of the currently edited file at point. By default, or with 1, insert filename of current buffer with complete absolute path.
	<code><f11> i F</code> <code><f6> F</code>	(pel-insert-filename-and-line & optional N)	Same as above but also append a colon and a line number, followed by a line end.  Useful for manually building a list of files inside a buffer. Later use the pel-open-at-point command M-<f6> (see 🔗 File-mngt) to open that file and move point to that line.
	<code><f11> i M-f</code> <code><f6> M-f</code>	(pel-insert-filename-wtilde & optional N)	Same as first command above, except that if the file is located in the current user home, insert the Unix-style tilde character ~ in place of the user home directory name.
	<code><f11> i C-f</code> <code><f6> C-f</code>	(pel-insert-dirname & optional N USE-TILDE)	Insert the directory path name of the currently edited file at point. By default, or with 1, insert directory name of file in current buffer.
If file is in user home, use ~ at the beginning	<code><f11> i C-M-f</code> <code><f6> C-M-f</code>	(pel-insert-dirname-wtilde & optional N)	Same as the above command, except that <i>if the file is located in the current user home</i> , insert the Unix-style tilde character ~ in place of the user home directory name.
	<code><f11> ? F</code>	(pel-show-function & optional INSERT-IT)	Display the name of the current “function” at point in the mini-buffer. See also: 🔗 Help/Info With any argument, like C-u , also insert the “function” name at point.
Insert software license	 The lice command, requires the lice external package  PEL activates it if pel-use-lice user option is t.		
Insert software license text	<code><f11> i L</code> <code><f6> L</code>	(lice NAME)	Insert license and headers at point. Prompts for license NAME, which is a license template name like “mit”, “gpl-3.0”, etc... Support TAB completion to get the complete list of templates.
Insert SPDX software license	<code><f11> i M-1</code> <code><f6> M-1</code> C-c i 1	(spdx-insert-spdx)	Insert a SPDX license header. Prompts for the SPDX license type. Supports tab completion.  Requires spdx activated by  pel-use-spdx user option.
Insert to-do note	The default to-do note format is: <code>[:todo (DATE) , by (USER):]</code> Automatically inserts the date and user name. Other popular prefix TODO :		
Insert To-Do note	<code><f11> i n</code> <code><f6> n</code>	(pel-insert-todo-note & optional UTC)	Insert a to-do note template comment. The format is configured by the pel-todo-note-text and the pel-todo-note-date-format user options.
	If “(USER)” is in pel-todo-note-text, it is replaced by the user name. If it has “(DATE)”, it is replaced by the date using the format specified pel-todo-note-date-format. By default the local date is entered. Use the C-u prefix to use UTC instead. Point is placed one character before the end of the inserted text.		
Customize to-do note format	<code><f11> i <f4></code> n	(pel-customize-todo-note)	Open the customize buffer to change the to-do note format user options.

Description	Keystroke	Function	Note																				
Insert date & time All formats are customizable You can modify them temporarily or permanently to fit your needs.	The following commands insert time stamps of specific formats: <table><tr><th>Format</th><th>Date only</th><th>Date & Week-day</th><th>Date & Time</th><th>Date, Week-day & Time</th></tr><tr><td>User-selected format</td><td><f11> i d</td><td><f11> i D</td><td><f11> i t</td><td><f11> i T</td></tr><tr><td>short format</td><td></td><td><f11> i C-d</td><td></td><td><f11> i C-t</td></tr><tr><td>ISO-8601 format</td><td><f11> i M-d</td><td><f11> i M-D</td><td><f11> i M-t</td><td><f11> i M-T</td></tr></table> Each of these commands insert the local date/time by default. Use the C-u prefix to insert the UTC date/time instead. User-selected formats are specified by custom variables. Use <f11> i <f4> to access the relevant customize buffer. The docstring show the string generated by the format selected at the time PEL was last byte-compiled.			Format	Date only	Date & Week-day	Date & Time	Date, Week-day & Time	User-selected format	<f11> i d	<f11> i D	<f11> i t	<f11> i T	short format		<f11> i C-d		<f11> i C-t	ISO-8601 format	<f11> i M-d	<f11> i M-D	<f11> i M-t	<f11> i M-T
Format	Date only	Date & Week-day	Date & Time	Date, Week-day & Time																			
User-selected format	<f11> i d	<f11> i D	<f11> i t	<f11> i T																			
short format		<f11> i C-d		<f11> i C-t																			
ISO-8601 format	<f11> i M-d	<f11> i M-D	<f11> i M-t	<f11> i M-T																			
Customize date/time format	<f11> i <f4> d	(pel-customize-insert-date-time)	Open the customize buffer to change the date/time insertion, allowing you to change the formats used for each of the following commands.																				
Insert current date	<f11> i d	(pel-insert-current-date &optional UTC)	Insert current date (only, no week-day, no time) at point. Like: 2023-10-15 Local by default, UTC if C-u prefix used.																				
Insert current date and week day	<f11> i D	(pel-insert-current-date-wkd &optional UTC)	Insert current date and week-day (no time) at point. Like: Sunday, October 15 2023 Local by default, UTC if C-u prefix used.																				
Insert current date and week day - short form	<f11> i C-d	(pel-insert-current-date-wkd-short &optional UTC)	Insert short format of current date and week-day (no time) at point. Like: Sun Oct 15 2023 Local by default, UTC if C-u prefix used.																				
Insert current date & time	<f11> i t	(pel-insert-current-date-time &optional UTC)	Insert current date and time at point. Like: 2023-10-15 13:20:24 EDT Local by default, UTC if C-u prefix used.																				
Insert current date, week-day & time	<f11> i T	(pel-insert-current-date-time-wkd &optional UTC)	Insert current date, week-day and time at point. Like: Sunday, October 15 2023 at 13:20:24 EDT Local by default, UTC if C-u prefix used.																				
Insert current date, week-day & time - short form	<f11> i C-t	(pel-insert-current-date-time-wkd-short &optional UTC)	Insert short format of current date, week-day and time at point. Like: Sun Oct 15 2023 at 13:20:24 EDT. Local by default, UTC if C-u prefix used.																				
Insert current ISO 8601 date	<f11> i M-d	(pel-insert-current-iso-date &optional UTC)	Insert ISO 8601 conforming abbreviated YYYY-MM-DD format date. Like: 2023-10-15 Local by default, UTC if C-u prefix is used.																				
Insert current ISO 8601 date and week day	<f11> i M-D	(pel-insert-current-iso-date-wkd &optional UTC)	Insert ISO 8601 conforming abbreviated week-day format like: 2023-10-15-W41-7 Local by default, UTC if C-u prefix is used.																				
Insert current ISO 8601 date & time	<f11> i M-t	(pel-insert-current-iso-date-time &optional UTC)	Insert ISO 8601 conforming abbreviated date/time/zone format like: 2023-10-15T13:20:24-0400 Local by default, UTC if C-u prefix is used. Example: 2023-10-15T17:20:24+0000																				
Insert current ISO 8601 date, week-day & time	<f11> i M-T	(pel-insert-current-iso-date-time-wkd &optional UTC)	Insert ISO 8601 conforming abbreviated week-day format, like: 2023-10-15T13:20:24-0400 W41-7 Local by default, UTC if C-u prefix is used. Example: 2023-10-15T17:20:24+0000 W41-7																				
Automatic File Time Stamp on file save	Emacs has a built-in automatic time-stamping of files. It must be activated by adding the time-stamp function to the before-save-hook variable. This can either be done via Emacs customization system or explicitly inside your init file with the following code: (add-hook 'before-save-hook 'time-stamp) The time stamp will be added to files that contain, inside their first 8 lines, a line that looks like one of the following: Time-stamp: <> Time-stamp: " " You can, however change these defaults and get Emacs to update all sorts of time stamp formats, even inside source code statements: Emacs controls automatic insertion of timestamp with the following variables: time-stamp-pattern consists of 4 parts, each one controlled by a variable: time-stamp-line-limit : identifies where in the file the time stamp can be located. Defaults to 8: the first 8 lines. time-stamp-start : identifies the text pattern that precedes the time stamp. time-stamp-end : identifies the end of the time stamp. time-stamp-format specifies the format of the time stamp. Something like "%:y-%02m-%02d %02H:%02M:%02S %u" to specify the date and time in ISO format, with the user login's name. time-stamp-time-zone specifies the time zone selection: nil : Emacs local time t : Universal time wall : system wall clock time TZ : controlled by a TZ environment variable The time-stamp-format and time-stamp-time-zone variables can be set in your init file or via the Emacs customization system. They are defined in the time-stamp customization group. You can, however change these defaults and get Emacs to update all sorts of time stamp formats, even inside source code statements: By default, the time-stamp string must be placed within the first 8 lines of the file, otherwise it will not be updated automatically. If you want it located somewhere else in your file set the time-stamp-line-limit file local variable. PEL provides the extra user-option to control the automatic generation of time-stamps: pel-update-time-stamp user-option controls whether time-stamps are automatically update time stamps in all files where a valid time-stamp corresponding to Emacs settings as described above. Set it to t (the default) to allow automatic time stamp updates. Set it to nil to prevent them. You can also toggle it globally for the current editing session by using the <f11> f M-t key sequence. To insert a non-updatable time stamp, the PEL package provides a set of text insert commands which include inserting a time stamp .																						
Update file time stamp	<f11> f t	(time-stamp)	Force update the time stamp string(s) in the current buffer. Updates a time stamp of format recognized by <i>Emacs current settings</i> even when automatic time-stamp update is off. More information about the “<i>Emacs current settings</i>” in the description block above.																				
See also: File mngt																							
Toggle time stamp automatic update	<f11> f M-t	(time-stamp-toggle-active &optional ARG)	Toggle ‘time-stamp-active’, setting whether <f11> f t updates a buffer. With ARG, turn time stamping on if and only if arg is positive.																				
Inserting & Automatically Updating Copyrights	Two Emacs built-in commands, shown below, support insertion and update of copyright notices inside files. Activate automatic updates of copyright by adding the copyright-update function to the list of before-save-hook variable with the following code: (add-hook 'before-save-hook 'copyright-update) Ensure that the copyright-limit user-option is large enough to allow automatic updating of copyright notices. PEL improves on this: The pel-update-copyright user-option activates automatic copyright update on save. The pel-skip-copyright-in user-option identifies files and directory trees where copyright updates are not done. The pel-copyright-noprompt-in user-option identifies files and directory trees where copyright updates done automatically does not prompt. PEL provides the pel--update-copyright function used as the before-save-hook when pel-update-copyright user-option is turned on. That function prevent copyright updates of files identified by the pel-skip-copyright-in user-option and honours a state, controlled by the pel-toggle-copyright-on-save command, where copyright update can be disabled even when pel-update-copyright is turned on.																						
Insert copyright notice	<f11> i c	(copyright &optional STR ARG)	Insert a copyright by \$ORGANIZATION notice at cursor. If the ORGANIZATION environment variable is not available, Emacs prompts for it.																				
Update file's copyright notice	<f11> i M-c	(copyright-update &optional ARG INTERACTIVEP)	Update copyright notice to indicate the current year. With prefix ARG, replace the years in the notice rather than adding the current year after them. If necessary, and 'copyright-current-gpl-version' is set, any copying permissions following the copyright are updated as well.																				
Only update exiting notice. Does not create one if it's missing.	copyright-update does not warn if there is no copyright in the current buffer to update. It does not create a missing notice.																						
Toggle copyright update on save	<f11> M-@	(pel-toggle-update-copyright-on-save &optional GLOBALLY)	Toggle copyright update on file save and display current state. By default change behaviour for local buffer only. When GLOBALLY argument is non-nil, using any prefix argument, change it for all buffers for the current Emacs editing session (the change does not persist across Emacs sessions). To modify the global state permanently modify the customized value of the 'pel-update-copyright' user option via the 'pel-pkg-for-filemng' group customize buffer with <f11> f <f2> 1.																				
This command is only available when the pel-update-copyright is set to t.																							

Description	Keystroke	Function	Note												
Insert Commented Lines	The following commands help insert commented lines or just underlines the current line of text using the character corresponding to one of the adornment level used for reStructuredText sections. The strings are commented according to the major mode of the current buffer. If the buffer has no identified comment strings, the command prompts for them the first time it is used in that type of buffer. The following commands are also listed in the ☞ Comments table.														
Insert commented line See also: • ☞ Comments • ☞ Filling/Justification	• <f11> i 1 • <f6> 1	(pel-insert-line &optional LINELEN)	Insert a (commented) line before/at current line. • If point is at the beginning of the line insert it there. • If point is in the middle of a line, move point at beginning of line before inserting it. • The number of dash characters of the line is specified by LINELEN: • If LINELEN is not specified the buffer's fill-column value is used. • It supports several programming and markup language and uses the comment style identified by the file extension. If the comment style is unknown the command prompts for one. ▀ fill-column is customizable and can be used as a file or directory variable.												
Comment-underline current line with level adornment 1-9	<f11> _ ☺	(pel-commented-adorn-1)	Insert a commented level-x reST line adornment at point. • ☺ := 1 to 9 for levels 1 to 9												
Comment-underline current line with level 10 adornment	<f11> _ 0	(pel-commented-adorn-10)	Insert a commented level-10 reST line adornment at point.												
Smart Dash Mode	• Anyone that has been writing Lisp code for a while knows that using dash as word separator instead of underscore is more natural and faster to type. Unfortunately most programming languages (all non-Lisp?) have restrictions on the characters available in identifiers and underscore is often used. Typing underscore requires hitting the Shift key and it annoys some people that enjoyed writing Lisp code. This is where the smart-dash-mode helps. You can insert underscore in text by typing the dash key without hitting the Shift key! A very useful mode. • More information is available in the author's page. 📦 Requires the smart-dash external package.  PEL activates it when pel-use-smart-dash is set to t.  To activate smart-dash-mode automatically: • for major modes supported by PEL, add smart-dash-mode to the pel-<MODE>activates-minor-mode user-option for the specific mode. • for other modes, add the mode name to the pel-modes-activating-smart-dash-mode user-option.														
Toggle smart-dash mode See also: • ☞ Numkeypad • ☞ Text Modes • ☞ Mode Line	<f11> i -	(smart-dash-mode &optional ARG)	Toggle the smart-dash-mode on/off. • When smart-dash-mode is active, it redefines the dash key to insert an underscore within C-style identifiers and a dash otherwise. This allows you to type all_lowercase_c_identifiers as comfortably as you would lisp-style-identifiers. • While Smart-Dash mode is active, you can type C-q - or use the minus key on the numeric keypad to override it and insert a dash after a C-style identifier character. You might need to do this if you want to type a cramped-looking expression like x-5. • If Smart-Dash mode is activated while in a C-like mode (c-mode, c++-mode, and objc-mode by default, customizable with 'smart-dash-c-modes') it will also activate Smart-Dash-C mode, which translates "_>" into "->" and "__" into "--" automatically so that struct pointer member access and postfix-decrement aren't made more difficult by Smart-Dash mode's tendency to insert underscores at the tail ends of identifiers whether you want it to or not. Note that this will necessitate that you type literal underscores if you want more than one underscore in a row. ⚠ Normally when smart-dash-mode is active the numeric dash key (<kp-subtract>) acts as a smart-dash only. 👉 However, with PEL, the behaviour of the keypad '-' is only partly affected when the smart-dash-mode is active and it depends on the Numlock state: • In Numlock OFF:<table><tr><td>with no marked area:</td><td>insert a dash. Numeric argument for multiple insertion is not supported.</td></tr><tr><td>with an area marked:</td><td>kill marked area</td></tr><tr><td>with area marked with er/expand-region:</td><td>kill marked area</td></tr></table> • In Numlock ON:<table><tr><td>with no marked area:</td><td>insert an underscore after letter, number or underscore, dash otherwise</td></tr><tr><td>with area marked normally:</td><td>Ignore the marked area; insert a dash at point</td></tr><tr><td>with area marked with er/expand-region:</td><td>Reduces the marked area semantically as controlled by er/expand-region</td></tr></table> For more information on the NumLock control and key support, see ☞ Numkeypad. 👉 With PEL type <f11> t m ? to display the status of text modes including dash-mode. 👉 With PEL, when pel-use-delight is turned on, a short lighter of a green dash is showing in the mode line when smart-dash-mode is active.	with no marked area:	insert a dash. Numeric argument for multiple insertion is not supported.	with an area marked:	kill marked area	with area marked with er/expand-region:	kill marked area	with no marked area:	insert an underscore after letter, number or underscore, dash otherwise	with area marked normally:	Ignore the marked area; insert a dash at point	with area marked with er/expand-region:	Reduces the marked area semantically as controlled by er/expand-region
with no marked area:	insert a dash. Numeric argument for multiple insertion is not supported.														
with an area marked:	kill marked area														
with area marked with er/expand-region:	kill marked area														
with no marked area:	insert an underscore after letter, number or underscore, dash otherwise														
with area marked normally:	Ignore the marked area; insert a dash at point														
with area marked with er/expand-region:	Reduces the marked area semantically as controlled by er/expand-region														
Smartparens Mode • Smartparens manual	Simplify insertion of matching pairs with the smartparens minor mode. PEL binds a set of keys, described below, to toggle activation of that mode. 📦 This uses the smartparens external package.  PEL activates it when pel-use-smartparens is set to t. • Smartparens enhances the behaviour of certain keys, namely those that are part of any pair or tag. • Mode line lighter: smartparens-mode: SP smartparens-strict-mode: SP/s														
Help on smartparens	<f11> (?	(sp-cheat-sheet &optional ARG)	Generate a cheat sheet of all the smartparens interactive functions. Shows inside Emacs buffer. • Without a prefix argument, print only the short documentation and examples. • With non-nil prefix argument ARG, show the full documentation for each function. • You can follow the links to the function or variable help page. • To get back to the full list, use M-x help-go-back. • You can use 'beginning-of-defun' and 'end-of-defun' to jump to the previous/next entry. • Examples are fontified using the 'font-lock-string-face' for better orientation.												
Describe user system	<f11> (M-?	(sp-describe-system STARTERKIT)	Describe user's system. Prompt for starter kit: Evil, Spacemacs, Vanilla. • The output of this function can be used in bug reports.												
Toggle smartparens mode	<f11> ((	(smartparens-mode &optional ARG)	Toggle smartparens mode.												
Toggle smartparens-strict mode	<f11> ()	(smartparens-strict-mode &optional ARG)	Toggle the strict smartparens mode. • When strict mode is active, 'delete-char', 'kill-word' and their backward variants will skip over the pair delimiters in order to keep the structure always valid (the same way as 'paredit-mode' does). This is accomplished by remapping them to 'sp-delete-char' and 'sp-kill-word'. There is also function 'sp-kill-symbol' that deletes symbols instead of words, otherwise working exactly the same (it is not bound to any key by default). • When strict mode is active, this is indicated with "/s" after the smartparens indicator in the mode list												
Toggle smartparens mode	<f11> (M-(	(smartparens-global-mode &optional ARG)	Toggle Smartparens mode in all buffers. • With prefix ARG, enable Smartparens-Global mode if ARG is positive; otherwise, disable it. • Smartparens mode is enabled in all buffers where 'turn-on-smartparens-mode' would do it.												
Toggle smartparens-strict mode	<f11> (M-)	(smartparens-global-strict-mode &optional ARG)	Toggle Smartparens-Strict mode in all buffers. • With prefix ARG, enable Smartparens-Global-Strict mode if ARG is positive; otherwise, disable it. • Smartparens-Strict mode is enabled in all buffers where 'turn-on-smartparens-strict-mode' would do it.												

Description	Keystroke	Function	Note
Text and code skeletons tempo skeletons	Several mechanisms have been developed to allow easy insertion of predefined text in Emacs. Emacs provides the built-in skeleton mechanism and the tempo skeletons.		
Generic skeletons	PEL supports both. They are used a little bit differently. PEL provides key bindings to the tempo skeletons: the generic code templates, accessible via the <f6> prefix key, and the language-specific code templates, accessible via the <f12> key prefix. PEL provides generic tempo skeletons as well as some specialized for specific programming languages. The generic skeletons are less powerful but often good enough for most types of files. They support all types of files recognized by Emacs as long as Emacs understands the way comments work for the file type which is normally the case. If Emacs does not know the file type the commands assume the file uses a comment start only and will prompt for that string.		
ⓘ Customize PEL Text Insertions control	<f6> <f2>	(pel-customize-pel &optional OTHER-WINDOW)	Customize PEL generic tempo skeleton customization groups that control the format of the various skeletons including the generic skeleton used by the <f6> h key (se below). If OTHER-WINDOW is non-nil (use C-u) , display in other window.
Insert generic file module header block — Language agnostic	<f6> h	(pel-generic-file-header)	Insert a file header block at the top of the file. Works only for buffer visiting a file.
After inserting the template, navigate through areas that must be filled with: - tempo-forward-mark: C-c . - tempo-backward-mark: C-c , See examples in manual	Supports all text file types. - Supports all programming and markup language files that have a dedicated major mode. It is also available in buffers for major modes explicitly supported by the <f12> <f12> key prefix. This way, those modes can use two different commands to insert file header blocks, each having its own different format. - It supports several programming and markup language and uses the comment style identified by the file extension. If the comment style is unknown the command prompts for one. - The layout of the entered text is controlled by user options. It is possible to create a user-specified skeleton this command will used instead of the one provided by PEL. 📌 Specify the format of the header via the user-options in the pel-pkg-generic-code-style customization group accessible via <f6> <f2> - The files that have no extensions are often used in Unix-like OS shell scripts. These files are also supported as Emacs can recognize them if they are stored in a bin directory. PEL also has special support for them and is controlled by the pel-sh-script-skeleton-control customization group, which is accessible as a child of the main group. 📌 After inserting the template you can use the tempo-forward-mark and tempo-backward-mark to move point to the beginning of each section that must be filled. ⚠️ The command key binding <f6> h is available only 1 second after Emacs has started.		
Toggle pel-tempo-mode	<f6> SPC	(pel-tempo-mode &optional ARG)	Toggle PEL tempo mode on/off. PEL tempo mode activates C-c . and C-c , , as well as to C-c C-. and C-c C-, , key bindings to navigate across tempo mark hot-spots. When pel-tempo-mode is active the pel-tempo-mode lighter (⚡) is shown on the status bar. The second set of keys are only available when Emacs runs in graphics mode. 📌 The pel-generic-file-header command inserts the text using a tempo skeleton: the PEL tempo mode is automatically activated by typing <f6> h .
Jump to next tempo mark	C-c M-f C-c . C-c C-.	(tempo-forward-mark)	Jump to the next mark in ‘tempo-back-mark-list’: the location where code must be updated inside the inserted skeleton. These key key bindings are only available when pel-tempo-mode is active.
Jump to previous tempo mark	C-c M-b C-c , C-c C-,	(tempo-backward-mark)	Jump to the previous mark in ‘tempo-back-mark-list’: the location where code must be updated inside the inserted skeleton. These key binding are only available when pel-tempo-mode is active.
Store PEL code template settings in .dir-locals.el to fine-tune layout of files in a directory tree	📌 Emacs user options by default take effect globally. But by using file and directory variables (see ⓘ File/Directory Variables) they can also be used to take effect on a single file or all files inside a directory tree. So by default, the user options that control the PEL tempo template take effect globally. If you want to change the behaviour for only one file, write the user option control block at the end of that file. If you want to control the behaviour of the PEL tempo templates for all files inside a directory tree create a .dir-locals file and store the values of the relevant options variables inside that file. This allows you to control the user options affecting the format of the tempo templates precisely.		
Example:	Although the default settings of pel-generic-skel-module-section-titles identifies the 3 sections “Module Description”, “Dependencies” and “Code” you can keep this for other files but inside a directory you can force all shell-mode files to use 2 sections: “Description” and “Script” and ensure that all files have a 1-line copyright notice with the .dir-locals.el file containing the following code: <pre>;;; Directory Local Variables ;;; For more information see (info "(emacs) Directory Variables") ((nil . ((pel-generic-skel-with-copyright . t) (pel-generic-skel-with-license . "MIT"))) (sh-mode . ((pel-generic-skel-module-section-titles . ("Description" "Script")))))</pre>		
Entering Templated Text with Tempo Skeletons See also: - Major mode specific: ⚡ - C ⚡ - C++ ⚡ - Emacs Lisp ⚡ - Erlang ⚡ reStructuredText	Emacs built-in support includes the tempo skeletons . PEL implements extension to the tempo skeleton Emacs built-in package under two prefix keys: - The commands under the <f6> prefix keys insert template text that are adapted to each major mode. They are generic in nature, and dynamically adapt to the major mode and the comment style supported by the major mode. The layout of the templates is the same for every major mode, they differ only by the comment strings. - The commands under the <f12> <f12> prefix key insert templates specialized for the programming or markup language of the major mode that support this key prefix. PEL attempts to use the same key bindings for equivalent concepts (such as file header block) inside each mode specific instance of the <f12> <f12> key maps as much as possible. The tempo skeletons provided by PEL can be quite complex and their formats are controlled by user options. PEL currently only support this key prefix with for the following major modes (more are planned): - C, C++, Emacs Lisp, Erlang - reStructuredText		
Major-mode specific Tempo Templates Prefix	<f12> <f12>	Key prefix sequence to the list of tempo skeleton commands. - This command prefix is available only for some major modes (see the list in the first column) of the section row above. - The commands under this prefix insert text specialized for their specific major mode, as opposed to the commands bound to the <f6> prefix key. For more information see the language specialized reference table.	

Description	Keystroke	Function	Note
Entering Templated Test with Yasnippet • See also:Y Customize Activate a yasnippet by identifying the directory holding the snippet files. ➡	PEL also supports the popular yasnipppet external package which provides another way to insert templated text, and yasnipppet-snippets external package which provides a large set of code snippets for a large set of major modes. • To use yasnippets, you must type the snippet abbreviation and then hit the TAB key to expand the text. 📦 Requires yasnipppet  activated when pel-use-yasnippet is set to t or to use-from-start. 📦 Requires yasnipppet-snippets  activated when pel-use-yasnippet-snippets is set to t. • Use the key <f11> i <f2> to access the PEL Insertion customization buffer to customize these user options. ⚠️ For yasnippet to work and find your snippets you must identify the directories holding the snippets in the yas-snippet-dirs user-option. • The name of the directories identified in the list must have the name snippets and must hold sub-directories that are named after the major mode. These sub-directories hold the snippet files that will become available in the major mode (and their derived modes). • Type <f11> y <f3> to open the yasnippet customization buffer and access that user-option. When the yasnippet-mode is enabled, the list of snippets available in the current buffer is: • listed in the menu bar Yasnippet menu entry (see Y Menus) and • can also be listed using the yas-describe-tables command (which PEL binds to <f11> y t). PEL binds the following yasnippet commands to keys in the pel: key prefix, shown below.		
Y Customize PEL yasnippet use	<f11> y <f2>	(pel-customize-pel &optional OTHER-WINDOW)	Customize PEL Yasnippet text insertion support. • If OTHER-WINDOW is non-nil (use C-u) , display in other window.
Y Customize Emacs yasnippet control	<f11> y <f3>	(pel-customize-library &optional OTHER-WINDOW)	Customize Yasnippet groups: yasnippet, yasnippet-snippets, yas-minor
Prints Yasnippet version info	<f11> y ?	(yas-about)	Prints yasnippet version information in the mini buffer.
Toggle YASnippet minor mode on/off	<f11> y y	(yas-minor-mode &optional ARG)	Toggle YaSnippet mode.
	• When YASnippet mode is enabled, ‘yas-expand’, normally bound to the TAB key, expands snippets of code depending on the major mode. • With no argument, this command toggles the mode. Positive prefix argument turns on the mode. Negative prefix argument turns off the mode. • YASnippet mode key bindings: keybinding ----- C-c & C-nyas-new-snippet C-c & C-syas-insert-snippet C-c & C-vyas-visit-snippet-file		
Toggle YASnippet global mode on/off	<f11> y Y	(yas-global-mode &optional ARG)	Toggle Yas minor mode in all buffers. • With prefix ARG, enable Yas-Global mode if ARG is positive; otherwise, disable it.
Display all snippets for current major mode	<f11> y t	(yas-describe-tables &optional WITH-NONACTIVE)	Display snippets for each table. Each table is associated with a major mode (and all the major-modes that derived from it). The tables show the group, the state and name, the abbreviation key(s) and bindings (if any). The names are button that open the actual snippet file.
Expand snippet whose name is just before point	TAB	(yas-expand &optional FIELD)	Expand a snippet before point. If no snippet expansion is possible, do nothing. • This key binding is only active when the YASnippet mode is active. Once the snippet was expanded the TAB key normal behaviour is restored.
Write a new snippet	• <f11> y n	(yas-new-snippet &optional NO-TEMPLATE)	Pops a new buffer for writing a snippet. • Expands a snippet-writing snippet, unless the optional prefix arg NO-TEMPLATE is non-nil.
	• C-c & C-n		
Prompt for snippet & insert it	• <f11> y s	(yas-insert-snippet &optional NO-CONDITION)	Choose a snippet to expand, pop-up a list of choices according to ‘yas-prompt-functions’. • With prefix argument NO-CONDITION, bypass filtering of snippets by condition.
	• C-c & C-s		
Visit a snippet file	• <f11> y v	(yas-visit-snippet-file)	Choose a snippet to edit, selection like ‘yas-insert-snippet’. • Only success if selected snippet was loaded from a file. Put the visited file in ‘snippet-mode’.
	• C-c & C-v		

Inserting Text — References

Topic & link	Description
GNU Emacs Manual: Time Stamps	
Smart-Dash Mode homepage	A description of this extremely useful mode and why it was created.